

02-Java集合

Java 集合面试题

1. 集合体系

问：Java 集合体系结构？

答：

```
Collection (单列)
├── List (有序可重复)
│   ├── ArrayList    动态数组
│   ├── LinkedList   双向链表
│   └── Vector        线程安全 (已过时)
├── Set (无序不重复)
│   ├── HashSet
│   ├── LinkedHashSet 保持插入顺序
│   └── TreeSet       红黑树, 有序
└── Queue
    ├── PriorityQueue 堆实现
    └── Deque          双端队列

Map (键值对, 独立于 Collection)
├── HashMap
├── LinkedHashMap
├── TreeMap
├── Hashtable         线程安全 (已过时)
└── ConcurrentHashMap 线程安全
```

问：数组和集合有什么区别？

答：

对比	数组	集合
长度	固定	动态
元素类型	基本类型 + 对象	只能存对象 (基本类型需包装类)
访问	下标直接访问	迭代器 / API
功能	简单	丰富 (排序、去重、线程安全实现等)

问：Collection 和 Collections 的区别？

答：– **Collection**：接口，所有单列集合的根（List、Set、Queue） – **Collections**：工具类，提供静态方法（sort、reverse、synchronizedList、emptyList 等）

问：Java 中有哪些线程安全的集合？

答：java.util（遗留）：Vector、Hashtable（已不推荐）

java.util.concurrent（推荐）：

类型	类
List	CopyOnWriteArrayList
Set	CopyOnWriteArraySet、ConcurrentSkipListSet
Map	ConcurrentHashMap、ConcurrentSkipListMap
Queue	ConcurrentLinkedQueue、BlockingQueue 系列

2. List

问：ArrayList 和 LinkedList 的区别？

答：

对比	ArrayList	LinkedList
底层	动态数组	双向链表
随机访问	O(1)	O(n)
头尾插入删除	尾 O(1)，头 O(n)	O(1)
内存	连续，缓存友好	节点分散，额外指针开销
线程安全	否	否
扩容	1.5 倍	无需扩容

选型：查询多用 ArrayList；频繁头尾增删用 LinkedList。

问：ArrayList 扩容机制？

答：1. 默认初始容量 10（首次 add 时分配） 2. 容量不足时扩容为原来的 1.5 倍（oldCapacity + (oldCapacity >> 1)） 3. 扩容时 Arrays.copyOf 复制到新数组

JDK 9+：扩容逻辑提取到 ArraysSupport.newLength，1.5 倍策略不变。

问：ArrayList 和 Vector 的区别？

答：

对比	ArrayList	Vector
线程安全	否	是（方法 synchronized）
扩容	1.5 倍	默认 2 倍（可设 capacityIncrement）
性能	单线程更快	同步开销大
推荐	是	已过时，用 CopyOnWriteArrayList 或 synchronizedList

问：ArrayList 线程安全吗？为什么不安全？如何变线程安全？

答：不安全。多线程并发 add 可能导致：1. 部分值为 null：两线程写到同一 index 2. 索引越界：size++ 非原子，两线程同时扩容判断失误 3. size 不准：size++ 丢失更新

变线程安全的方式：

```
Collections.synchronizedList(list); // 读写都加锁
CopyOnWriteArrayList<>();           // 写时复制，读多写少
new Vector<>();                       // 遗留方案，不推荐
```

问：List 可以一边遍历一边修改吗？

答：

方式	修改元素值	add/remove 结构
普通 for + set	✅ 可以	✅ 可以（注意 index 变化）
foreach	set 可以	❌ 抛 ConcurrentModificationException
Iterator	next 后用 iterator.remove()	✅ 仅迭代器 remove
fail-safe 集合	CopyOnWriteArrayList 可以	操作副本

原则：结构修改用迭代器 remove()，或普通 for 倒序删除。

问：List 和数组如何互相转换？

答：

```
// 数组 → List
List<String> list = Arrays.asList(arr);           // 固定大小，不能 add/remove
List<String> list2 = new ArrayList<>(Arrays.asList(arr)); // 可变

// List → 数组
String[] arr = list.toArray(new String[0]);
```

注意： `Arrays.asList()` 返回的 List 与原数组共享底层，修改会互相影响。

3. HashMap（最高频）

问：HashMap 底层结构？

答：JDK 8：数组 + 链表 + 红黑树

- 默认初始容量 16，负载因子 0.75
- 链表长度 > 8 且数组长度 ≥ 64 时，链表转红黑树
- 红黑树节点 < 6 时退化为链表
- 扩容为原来的 2 倍

问：HashMap put 流程？

答：1. 计算 `hash = (h = key.hashCode()) ^ (h >>> 16)` 扰动 2. 定位桶： `index = (n - 1) & hash` 3. 桶为空：直接放入 4. 桶不为空： – key 相同：覆盖 value – key 不同：尾插链表 / 红黑树插入 5. 元素数 > 容量 $\times$ 0.75：扩容 2 倍并 rehash

问：HashMap get 流程？

答：1. 计算 `hash(key)`，定位桶 `(n-1) & hash` 2. 桶为空 $\rightarrow$ 返回 null 3. 比较头节点：hash 相等且 `key equals` $\rightarrow$ 返回 value 4. 否则遍历链表或红黑树 `getTreeNode()` 查找 5. 未找到 $\rightarrow$ null

null key：hash 固定为 0，存在 0 号桶，只允一个 null key。

问：HashMap get 一定安全吗？

答：不一定： – Map 本身为 null $\rightarrow$ NPE – 多线程并发读写 $\rightarrow$ 可能读到错误数据或结构异常 – 正常单线程下 `get(null)` 合法，返回 null 表示 key 不存在或 value 为 null

并发场景用 `ConcurrentHashMap`。

问：哈希冲突有哪些解决方法？

答：

方法	说明	HashMap 使用
链地址法	冲突元素链在同一桶	✅ 链表 + 红黑树
开放寻址	探测下一个空槽	❌
再哈希	换 hash 函数重算	❌
扩容	扩大桶数量减少冲突	✅ loadFactor 触发

问：HashMap 为什么容量是 2 的幂？

答：使用 `(n - 1) & hash` 代替 `% n` 计算桶下标：- 位运算比取模快 - 2 的幂减 1 的二进制全为 1，使 hash 分布更均匀

问：HashMap 线程不安全的表现？

答：1. **JDK 7**：多线程扩容可能形成环形链表，导致 get 死循环 2. **数据覆盖**：并发 put 同一桶，后写入的覆盖先写入的 3. **size 不准确**：并发修改 size 计数

并发场景使用 `ConcurrentHashMap`。

问：HashMap 和 Hashtable 的区别？

答：

对比	HashMap	Hashtable
线程安全	否	是 (synchronized)
null key/value	允许一个 null key	不允许
效率	高	低
推荐	是	已过时

4. ConcurrentHashMap

问：ConcurrentHashMap 如何保证线程安全？

答：

JDK 7：分段锁 (Segment)，每个 Segment 继承 ReentrantLock，锁粒度为段。

JDK 8：- 取消 Segment，与 HashMap 结构类似 - **空桶**：CAS 插入 - **非空桶**：synchronized 锁桶头节点 - 扩容时多线程协助迁移

问：ConcurrentHashMap 为什么同时用 CAS 和 synchronized？

答：

场景	手段	原因
空桶插入	CAS	无竞争，CAS 开销小
非空桶操作	synchronized 锁头节点	需遍历链表/树，CAS 无法保证复合操作
初始化 table	CAS	仅一个线程成功初始化

权衡：低竞争用乐观锁，高竞争/复杂操作悲观锁。

问：ConcurrentHashMap 用了悲观锁还是乐观锁？

答：两者都有： – 乐观锁：空桶 CAS 插入、初始化、baseCount 更新 – 悲观锁：非空桶 synchronized 锁头节点

问：ConcurrentHashMap 和 Hashtable 的区别？

答：

对比	Hashtable	ConcurrentHashMap
锁粒度	整个 Map（方法 synchronized）	桶级别（JDK8）
null	不允许 null key/value	不允许 null key/value
效率	低	高
迭代	强一致	弱一致（不抛 CME）
推荐	否	是

问：ConcurrentHashMap 复合操作能保证原子性吗？

答：不能保证。size()、containsKey() + put() 等组合操作非原子。

解决： – putIfAbsent()、replace()、compute()、merge() 等原子 API（JDK 8+） – 或外部加锁

5. HashSet

问：HashSet 底层实现？

答：底层是 HashMap，元素存 key，value 为固定的 PRESENT 空对象。利用 HashMap 的 key 唯一性实现去重。

问：List 和 Set 的区别？

答：

对比	List	Set
有序	有序（插入顺序）	HashSet 无序；LinkedHashSet/TreeSet 有序
重复	允许	不允许
实现	ArrayList、LinkedList	HashSet、LinkedHashSet、TreeSet
访问	索引 get(i)	迭代器，无索引

问：HashSet、LinkedHashSet、TreeSet 的区别？

答：

对比	HashSet	LinkedHashSet	TreeSet
底层	HashMap	LinkedHashMap	TreeMap（红黑树）
顺序	无序	插入顺序	排序（自然/Comparator）
性能	O(1)	O(1)	O(log n)
null	允许一个 null	允许一个 null	不允许 null

问：Set 如何保证元素不重复？

答：1. add 时：先算 hashCode() 定位桶，再 equals() 比较 2. hash 不同 → 直接插入 3. hash 相同且 equals 相等 → 视为重复，不插入 4. hash 相同但 equals 不等 → 拉链/树存储（哈希冲突）
必须重写 equals 和 hashCode，且 equals 相等时 hashCode 必须相等。

问：如何对 Set 排序？

答：

方式	说明
TreeSet	天然有序，构造时传入 Comparator
List 中转	<code>new ArrayList<>(set) → Collections.sort()</code>
Stream	<code>set.stream().sorted().collect(toList())</code>

6. TreeMap / TreeSet

问：TreeMap 如何保证有序？

答：底层 红黑树（自平衡二叉搜索树），按 key 的自然顺序或自定义 Comparator 排序。增删改查 O(log n)。

7. 综合题

问：Comparable 和 Comparator 的区别？

答：

对比	Comparable	Comparator
位置	实现类内部	独立比较器
方法	compareTo()	compare()
排序	自然排序	定制排序
修改	需改源码	不改源码

问：fail-fast 和 fail-safe 的区别？

答：– fail-fast (ArrayList、HashMap)：迭代时检测 modCount，结构被修改抛 ConcurrentModificationException – fail-safe (CopyOnWriteArrayList)：迭代副本，不抛异常，但可能读到旧数据

8. HashMap 扩容详解

问：HashMap resize 扩容流程是怎样的？

答：触发条件：size > capacity × loadFactor (默认 0.75)。

JDK 8 扩容步骤：1. 新容量 = 旧容量 × 2，不超过 1 << 30 2. 创建新数组，遍历旧数组每个桶 3. 优化 rehash：节点在新数组的位置要么 原索引 i，要么 i + oldCap (看 hash 高位是 0 还是 1)，无需重新计算 hash 4. 链表/红黑树整体迁移到新桶

JDK 7 问题：头插法迁移链表可能形成环，多线程下 get 死循环。JDK 8 改为尾插法解决。

问：HashMap 为什么负载因子默认 0.75？

答：空间与时间的折中：– 太小 (如 0.5)：频繁扩容，浪费空间 – 太大 (如 1.0)：链表/树过长，查询变慢 – 0.75 是根据泊松分布统计，链表长度超过 8 的概率极低 (约 0.00000006)

问：HashMap 的 hash 扰动函数为什么要 (h ^ (h >>> 16))？

答：hashCode() 是 32 位，而桶索引只用低几位 (n-1) & hash。高位不参与运算会导致 哈希碰撞 集中在少数桶。

^ (h >>> 16) 将高 16 位混入低 16 位，使 hash 分布更均匀，减少链表长度。

9. ConcurrentHashMap 深入

问：ConcurrentHashMap put 流程详解？

答：1. 若数组未初始化，CAS 初始化 2. 计算 hash，定位桶下标 3. 桶为空：CAS 插入新节点，成功则 break 4. 桶不为空：– 若 hash == MOVED (-1)：说明正在扩容，帮助迁移 – 否则 synchronized 锁桶头节

点，再判断空/非空，插入链表或红黑树 5. 插入成功后 `size++`（用 LongAdder 思想分段计数，`baseCount + CounterCell[]`） 6. 若链表长度 ≥ 8 且数组长度 ≥ 64 ：树化 7. 若 `size` 超过阈值：触发扩容

关键：锁粒度是单个桶头节点，比 Hashtable 锁整个 Map 粒度更细。

问：ConcurrentHashMap 的 size 如何统计？

答：JDK 8 使用 `baseCount + CounterCell[]` 类似 LongAdder 的分段计数： - 低竞争：直接 CAS 更新 `baseCount` - 高竞争：分散到不同 `CounterCell`，减少 CAS 冲突 - `size()` 返回所有 `CounterCell` 之和 + `baseCount`

问：ConcurrentHashMap 扩容时多线程如何协助？

答：1. 只有一个线程创建新数组，设置 `transferIndex` 2. 其他线程从 `transferIndex` 向前领取一段桶（默认 16 个）进行迁移 3. 迁移完的桶头节点设为 `ForwardingNode`（`hash = MOVED`），其他线程看到 `MOVED` 会帮助迁移 4. 全部迁移完成后替换 `table` 引用

10. CopyOnWriteArrayList

问：CopyOnWriteArrayList 原理和适用场景？

答：写时复制（COW）： - 读：无锁，直接读底层数组 - 写：加 `ReentrantLock`，复制一份新数组，修改后替换引用

优点：读多写少场景读性能极高，迭代器 fail-safe。 **缺点：**写操作开销大（复制整个数组），内存占用翻倍，数据有短暂不一致。

适用：监听器列表、配置快照、黑白名单等读远多于写的场景。

问：CopyOnWriteArrayList 和 Collections.synchronizedList 怎么选？

答：

对比	CopyOnWriteArrayList	synchronizedList
读性能	无锁，极高	每次读也加锁
写性能	复制数组，差	加锁写入，较好
迭代	快照，不抛 CME	需手动同步
场景	读多写极少	读写均衡

11. TreeMap 红黑树

问：TreeMap 底层红黑树的特性？为什么用红黑树不用 AVL？

答：红黑树五大性质：1. 节点红或黑 2. 根节点黑 3. 叶子（NIL）黑 4. 红节点的子节点必黑（无连续红） 5. 任意节点到叶子的所有路径含相同黑节点数

vs AVL：AVL 严格平衡（左右子树高度差 ≤ 1 ），查询更快但插入删除旋转多；红黑树近似平衡，插入删除最多 3 次旋转，综合性能更优。

TreeMap 增删查均为 $O(\log n)$ ，按 key 有序，支持 `subMap`、`headMap`、`tailMap` 范围操作。

问：TreeMap 和 HashMap 如何选择？

答：

场景	选择
需要有序（自然序/自定义序）	TreeMap
追求 $O(1)$ 性能、无需有序	HashMap
需要插入顺序	LinkedHashMap
高并发	ConcurrentHashMap

TreeMap 不允许 null key（需比较大小，null 无法比较）。

12. 迭代器与 fail-fast

问：fail-fast 原理是什么？modCount 如何工作？

答：ArrayList、HashMap 等维护 `modCount`（结构修改次数）。迭代器创建时记录 `expectedModCount = modCount`。

每次 `next()` 检查 `modCount == expectedModCount`，不等则抛 `ConcurrentModificationException`。

触发结构修改的操作：add、remove、clear（直接调用集合方法，非迭代器自己的 remove）。

注意：`remove()` 通过迭代器调用时会同步更新 `expectedModCount`，不会抛异常。

问：HashMap 迭代顺序是什么？如何保证顺序？

答：HashMap 不保证顺序，迭代顺序取决于 hash 桶分布，可能随扩容变化。

需要顺序：– 插入顺序 → `LinkedHashMap`（双向链表维护顺序）– 排序 → `TreeMap` – 并发 + 快照读 → `CopyOnWriteArrayList` / `ConcurrentHashMap`（弱一致迭代）

13. LinkedHashMap

问：LinkedHashMap 如何实现 LRU 缓存？

答：继承 HashMap，额外维护 **双向链表** 记录插入/访问顺序。

```
new LinkedHashMap<>(16, 0.75f, true) // accessOrder = true
@Override
protected boolean removeEldestEntry(Map.Entry eldest) {
    return size() > MAX_CAPACITY;
}
```

`accessOrder = true` 时，get/put 会将节点移到链表尾部；超出容量时 `removeEldestEntry` 删除最久未使用的头节点。

14. PriorityQueue

问：PriorityQueue 底层实现？是线程安全的吗？

答：底层是 **小顶堆**（数组实现的完全二叉树），peek/poll $O(1)/O(\log n)$ ，offer $O(\log n)$ 。

非线程安全。并发场景用 `PriorityBlockingQueue`（基于 `ReentrantLock` + 堆）。

15. 其他高频

问：HashMap 的 key 可以用自定义对象吗？要注意什么？

答：可以，但必须 **同时重写 equals 和 hashCode**，且遵循：– equals 相等 → hashCode 必须相等 – hashCode 相等 → equals 不一定相等 – 用作 key 后 **不可变**（修改字段会导致 hash 变化，找不到值）

String、Integer 等不可变类是理想 key。

问：WeakHashMap 有什么特点？应用场景？

答：key 是 **弱引用**，GC 回收 key 后对应 entry 会被自动移除。

场景：缓存附加信息（如 ClassLoader 缓存）、ThreadLocal 底层 Map 的实现参考。防止 key 对象无法被 GC。

16. Map 遍历与选型

问：HashMap 有哪些遍历方式？

答：

```
// 1. entrySet (推荐, 可删元素)
for (Map.Entry<K,V> e : map.entrySet()) { ... }

// 2. keySet
for (K key : map.keySet()) { ... }

// 3. values
for (V val : map.values()) { ... }

// 4. forEach (JDK 8+)
map.forEach((k, v) -> ...);

// 5. Iterator
Iterator<Map.Entry<K,V>> it = map.entrySet().iterator();
```

性能：entrySet 一次遍历拿 key+value；keySet 遍历还要 get，多一次哈希查找。

问：往 HashMap 存 20 个元素（默认构造），扩容几次？

答：默认容量 16，负载因子 0.75，阈值 12。

- 第 1~12 个：不扩容
- 第 13 个：触发扩容 16 → 32
- 第 14~20 个：不扩容

共扩容 1 次。

17. BlockingQueue 与 Deque

问：BlockingQueue 是什么？常见实现？

答：阻塞队列，支持生产者-消费者模式：队列满时 put 阻塞，空时 take 阻塞。

实现	特点
ArrayBlockingQueue	有界数组，一把 ReentrantLock
LinkedBlockingQueue	可选有界，两把锁（头尾分离）
SynchronousQueue	不存储，直接交付（线程池 Cached）
PriorityBlockingQueue	无界堆，优先级排序
DelayQueue	延迟元素到期才能取

vs ConcurrentLinkedQueue：CLQ 无阻塞，高并发非阻塞；BlockingQueue 提供阻塞等待语义。

问：ArrayDeque 和 LinkedList 的区别？

答：

对比	ArrayDeque	LinkedList
底层	循环数组	双向链表
角色	栈/队列专用	List + Deque
性能	通常更快，缓存友好	节点开销大
null	不允许 null 元素	允许

推荐：作栈或队列优先 ArrayDeque，而非 LinkedList。

18. 集合遍历

问：集合有哪些遍历方式？

答：

方式	适用	注意
普通 for	List	可修改结构
foreach	Collection	结构修改抛 CME
Iterator	Collection	用 iterator.remove() 删
forEach / Stream	JDK 8+	函数式
ListIterator	List	双向遍历

大厂追问

1. HashMap 容量 16 时，最多存多少元素？

负载因子 0.75，最多 12 个元素触发扩容 ($16 \times 0.75 = 12$)。实际可存更多，但第 13 个会触发 resize。

2. 为什么 ConcurrentHashMap 不允许 null？

get(key) 返回 null 无法区分「key 不存在」和「value 就是 null」。多线程下这种歧义会导致逻辑错误，Doug Lea 设计时直接禁止 null key/value。

3. HashMap 树化条件为什么是 8 和 6?

泊松分布下，负载因子 0.75 时链表长度达到 8 的概率极低。设为 8 树化、6 退化，避免频繁树化/退化抖动。数组长度 < 64 时只扩容不树化。

4. 遍历 HashMap 删元素正确方式?

用迭代器 `iterator.remove()`，或 `entrySet().removeIf()`。foreach 中直接 `map.remove()` 会 fail-fast。

5. ConcurrentHashMap 1.7 和 1.8 最大区别?

1.7 分段锁（16 个 Segment，ReentrantLock 可重入），1.8 CAS + synchronized 锁桶。1.8 锁粒度更细、链表转红黑树、扩容多线程协助，并发度更高。

6. 为什么链表转红黑树不用 AVL?

AVL 严格平衡，插入删除旋转多；红黑树弱平衡，增删旋转少。HashMap 树化是低频场景（链表 ≥ 8 ），综合增删查性能红黑树更优。

7. HashMap 为什么 String 适合做 Key?

String 不可变，hashCode 缓存且不变，equals 可靠；作为 key 后修改内容会导致找不到 value。

8. LinkedList 真的比 ArrayList 插入快吗?

不一定。中间插入需 $O(n)$ 找位置；节点分配分散、缓存不友好。多数场景 ArrayList 更优，LinkedList 适合已知节点引用或频繁头尾操作。

9. CopyOnWriteArrayList 迭代器会抛 CME 吗?

不会。迭代器持有创建时的数组快照，写操作复制新数组，读不受写影响（弱一致，可能读到旧数据）。

10. HashMap 和 ConcurrentHashMap 初始容量怎么设?

已知元素数 n ，设 `initialCapacity = (int)(n / loadFactor) + 1`，再取大于等于该值的最小 2 的幂（HashMap 会自动调整）。减少扩容次数。